



Salidas estructuradas

Hasta aquí el modelo devolvía **texto libre** útil para humanos. En software, muchas veces necesitas **datos machine-readable**: listas, tablas, **JSON** con claves fijas para `json.loads`, ramificar con `if`, guardar en base de datos.

El LLM deja de ser solo “chatbot” y pasa a ser **componente** de tu pipeline — si pides formato y **validas** lo que vuelve.

Objetivos

- Diferenciar salida: texto libre, listas, JSON.
- Pedir JSON en la **tarea** y reforzarlo en la **config** de Gemini.
- Parsear y validar en Python (continuidad Unidad 1 y Sprint 4).
- Construir un clasificador de incidencias con salida estructurada.

1) Tipos de salida

Tipo	Uso	Procesamiento en Python
Texto libre	emails, explicaciones, resúmenes prose	<code>print</code> , mostrar UI
Lista / viñetas	pasos, recomendaciones	split heurístico o pedir JSON array
Tabla markdown	informes legibles	parsing frágil; mejor JSON si automatizas
JSON	clasificación, extracción, APIs internas	<code>json.loads</code> + validación de esquema

Para automatización seria, **JSON** (o esquema validado) es el estándar.

2) Pedir JSON en la tarea

```
TASK_CLASIFICAR = """
Tarea: clasifica el mensaje del usuario como incidencia de soporte.
```

Devuelve EXCLUSIVAMENTE un objeto JSON válido (sin markdown, sin texto extra) con estas claves:

- "category": string, una de: soporte, facturacion, ventas, otro
- "priority": string, una de: baja, media, alta
- "summary": string, resumen en una frase

Si el mensaje está vacío, devuelve:

```
{"category": "otro", "priority": "baja", "summary": "mensaje vacío"}
"""
```

Sin refuerzo en la API, el modelo a veces envuelve el JSON en ````json` o añade frases — por eso combinas con config (siguiente sección).

3) Refuerzo con Gemini: `response_mime_type`

Patrón ya visto en Sprint 4 ([03_IA_Generativa_con_python.md](#)):

```
import json
from google import genai
from google.genai import types

client = genai.Client()

role = "Eres un clasificador de tickets de soporte. Respondes solo con JSON."
mensaje = "No puedo acceder a mi cuenta desde ayer, necesito ayuda urgente."

task = """
Clasifica el mensaje. JSON con claves: category, priority, summary.
category: soporte | facturacion | ventas | otro
priority: baja | media | alta
"""

prompt = f"{role}\n\n{task}\n\nMensaje:\n{mensaje}"

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=prompt,
    config=types.GenerateContentConfig(
        temperature=0,
        response_mime_type="application/json",
    ),
)

raw = (response.text or "").strip()
```

Temperatura 0 (o baja) reduce creatividad no deseada en clasificación.

4) Parseo y validación en Python

```
CATEGORIAS = {"soporte", "facturacion", "ventas", "otro"}
PRIORIDADES = {"baja", "media", "alta"}

def parsear_clasificacion(raw: str) -> dict:
    try:
        obj = json.loads(raw)
    except json.JSONDecodeError as e:
        raise ValueError(f"JSON inválido del modelo: {raw!r}") from e

    if not isinstance(obj, dict):
        raise ValueError(f"Se esperaba objeto JSON, recibido: {type(obj)}")

    faltan = {"category", "priority", "summary"} - obj.keys()
    if faltan:
        raise ValueError(f"Faltan claves: {faltan}")

    if obj["category"] not in CATEGORIAS:
        raise ValueError(f"category inválida: {obj['category']}")
    if obj["priority"] not in PRIORIDADES:
        raise ValueError(f"priority inválida: {obj['priority']}")

    return obj
```

5) Envolver en `respuesta_ok` / `respuesta_error`

```
def clasificar_incidencia(mensaje: str) -> dict:
    if not mensaje.strip():
        return {
            "status": "error",
            "mensaje": "Mensaje vacío",
            "data": {"errores": ["El mensaje no puede estar vacío."]},
        }

    try:
        raw = llamar_gemini_clasificacion(mensaje) # función que devuelve str
        obj = parsear_clasificacion(raw)
        return {
            "status": "ok",
            "mensaje": "Clasificación completada",
            "data": obj,
        }
    except ValueError as e:
        return {
            "status": "error",
            "mensaje": "Salida del modelo inválida",
```

```
        "data": {"errores": [str(e)]},
    }
```

Tu `main` puede ramificar:

```
resultado = clasificar_incidencia("No puedo acceder a mi cuenta")
if resultado["status"] == "ok":
    print("Prioridad:", resultado["data"]["priority"])
else:
    for err in resultado["data"]["errores"]:
        print("-", err)
```

6) Ejemplo completo: clasificador de incidencias

```
import json
from google import genai
from google.genai import types

MODEL = "gemini-3-flash-preview"

ROLE = "Eres un analista de soporte. Devuelves solo JSON válido."

TASK = """
Clasifica el mensaje del usuario.
Claves obligatorias: category, priority, summary.
Valores permitidos:
- category: soporte, facturacion, ventas, otro
- priority: baja, media, alta
"""

def build_clasificacion_prompt(mensaje: str) -> str:
    return f"{ROLE}\n\n{TASK}\n\nMensaje:\n{mensaje.strip()}\n"

def llamar_gemini_json(prompt: str) -> str:
    client = genai.Client()
    response = client.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=0,
            response_mime_type="application/json",
        ),
    )
    return (response.text or "").strip()
```

```
def clasificar(mensaje: str) -> dict:
    prompt = build_clasificacion_prompt(mensaje)
    raw = llamar_gemini_json(prompt)
    return parsear_clasificacion(raw)

if __name__ == "__main__":
    casos = [
        "No puedo acceder a mi cuenta.",
        "¿Cuándo llega mi factura de marzo?",
        "Quiero comprar 50 licencias para mi equipo.",
    ]
    for msg in casos:
        print(f"\nEntrada: {msg}")
        try:
            print("Salida:", clasificar(msg))
        except ValueError as e:
            print("Error:", e)
```

Salida esperada (aproximada) para el primer mensaje:

```
{
  "category": "soporte",
  "priority": "alta",
  "summary": "Problema de acceso a la cuenta"
}
```

7) Texto libre con estructura ligera

Si no necesitas JSON estricto, puedes pedir **lista numerada** y parsear con cuidado:

```
task = """
Devuelve exactamente 3 líneas numeradas (1. 2. 3.).
Cada línea: una recomendación de máximo 12 palabras.
"""
```

Para producción, JSON suele ser más robusto que regex sobre viñetas.

8) Esquema en la instrucción (documentación para el modelo)

Incluir un "esqueleto" reduce errores de claves:

```
TASK = """
Devuelve JSON con esta forma:
{
```

```

"category": "",
"priority": "",
"summary": ""
}
Rellena los valores. Sin comentarios fuera del JSON.
"""

```

Algunos equipos usan **JSON Schema** en APIs avanzadas; para el bootcamp, validación manual en Python basta.

9) Cuando el modelo “rompe” el contrato

Situaciones habituales:

- Markdown alrededor del JSON → preprocesar:

```

def extraer_json(raw: str) -> str:
    s = raw.strip()
    if s.startswith("`"):
        lines = s.splitlines()
        lines = [ln for ln in lines if not ln.strip().startswith("`")]
        s = "\n".join(lines).strip()
    return s

```

- Claves en inglés cuando pediste español en valores → refuerza en TASK y whitelist.
- `priority: "urgent"` fuera de whitelist → `parsear_clasificacion` falla → `respuesta_error` y reintento o fallback humano.

Reintento (opcional):

```

def clasificar_con_reintento(mensaje: str, max_intentos: int = 2) -> dict:
    ultimo_error = None
    for _ in range(max_intentos):
        try:
            return clasificar(mensaje)
        except ValueError as e:
            ultimo_error = e
    raise ultimo_error

```

10) JSON + datos externos

Clasificar **con contexto** de política interna:

```

politica = {
    "prioridad_alta_si": ["sin acceso", "caída del servicio", "pago fallido"],

```

```

}

prompt = f"""
{ROLE}

Política de prioridad (referencia):
{json.dumps(politica, ensure_ascii=False, indent=2)}

{TASK}

Mensaje:
{mensaje}
"""

```

El JSON de **salida** sigue siendo **category** / **priority** / **summary**; el JSON de **entrada** es datos de referencia.

11) Función objetivo **build_prompt**

```

def build_prompt(
    role: str,
    task: str,
    user_input: str,
    additional_data: dict | None = None,
) -> str:
    parts = [role.strip(), "", task.strip()]
    if additional_data:
        parts.extend([
            "",
            "--- DATOS DE REFERENCIA ---",
            json.dumps(additional_data, ensure_ascii=False, indent=2),
            "--- FIN DATOS ---",
        ])
    parts.extend(["", "Entrada del usuario:", user_input.strip()])
    return "\n".join(parts)

```

Flujo completo:

```
build_prompt(...) → Gemini → raw → parsear / validar → respuesta_ok | error
```

12) Errores frecuentes

1. **Confiar en JSON sin `json.loads`** — bugs en producción.
2. **No definir whitelist de valores** — el modelo inventa categorías.
3. **Temperatura alta en clasificación** — respuestas inconsistentes.
4. **No manejar fallo de parseo** — el programa crashea.

5. Mezclar salida humana y máquina — decide uno u otro por endpoint.

13) Cierre de la unidad

Al terminar esta sección y la unidad entera:

- Sabes **controlar** al modelo con rol y tarea.
- Sabes **generar** prompts desde Python.
- Sabes **inyectar** datos de usuario y externos.
- Sabes **exigir** JSON y **validarlo** como cualquier otro input.

Siguiente unidad (Context Engineering):

“Ahora que sabemos construir prompts, vamos a decidir qué contexto merece la pena incluir dentro de ellos.”

Resumen

- Salida estructurada = puente entre LLM y el resto de tu código.
- Combina **instrucción clara**, **response_mime_type**, **json.loads**, **whitelist**.
- Clasificador de incidencias = caso canónico de esta sección.
- Siempre plan B cuando el modelo rompe el contrato (reintento, error legible).